

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and K-Nearest Neighbors Algorithms
GitHub Repository	https://github.com/Ray-Audrey/ML-Experiments
Student Name	Harshita S
Register Number	3122247001020
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	27/07/2026

1 Objective

The objective of this experiment is to develop machine learning models for classifying email messages as either spam or non-spam using the Spambase dataset. The experiment aims to implement and compare different variants of the Naïve Bayes classifier, namely Gaussian Naïve Bayes, Multinomial Naïve Bayes, and Bernoulli Naïve Bayes, along with the K-Nearest Neighbors (KNN) classifier. The models are evaluated using standard classification metrics, cross-validation, hyperparameter tuning, and computational time analysis.

2 Problem Statement

Electronic mail has become one of the most widely used communication platforms, making it a frequent target for unsolicited spam messages. Spam emails consume storage space, waste network resources, and may contain phishing attacks or malicious software. Therefore, automatic spam detection is an important application of machine learning.

In this experiment, the Spambase dataset is used to build supervised classification models capable of distinguishing spam emails from legitimate emails. The dataset undergoes exploratory data analysis and preprocessing before training multiple machine learning algorithms. The performance of Gaussian Naïve Bayes, Multinomial Naïve Bayes, Bernoulli Naïve Bayes, and K-Nearest Neighbors is evaluated and compared using classification metrics, confusion matrices, ROC curves, precision-recall curves, cross-validation, and hyperparameter optimization techniques.

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository
Number of Samples	4601
Number of Features	57
Target Variable	Class (Spam / Non-Spam)
Number of Classes	2
Missing Values	0
Duplicate Samples	391
Train-Test Split	80% Training, 20% Testing

Feature Selection Method	ANOVA (SelectKBest)
Scaling	StandardScaler (for Gaussian NB and KNN only)

4 Exploratory Data Analysis

The Spambase dataset was analyzed using a reusable Exploratory Data Analysis (EDA) function developed in Python. A consolidated EDA summary figure containing twelve subplots was generated to visualize different aspects of the dataset. These visualizations include dataset overview, descriptive statistics, missing value analysis, duplicate record analysis, class distribution, feature distributions, boxplots, pairwise relationships, correlation matrix, and distribution of important numerical attributes.

The EDA process provides valuable insights into the characteristics of the dataset before model training. It helps identify data quality issues, understand feature distributions, observe class imbalance, and determine whether preprocessing techniques such as scaling or encoding are required.

- Dataset overview
- Statistical summary
- Missing value analysis
- Duplicate record analysis
- Class distribution
- Correlation matrix
- Feature distribution

EDA Summary Figure

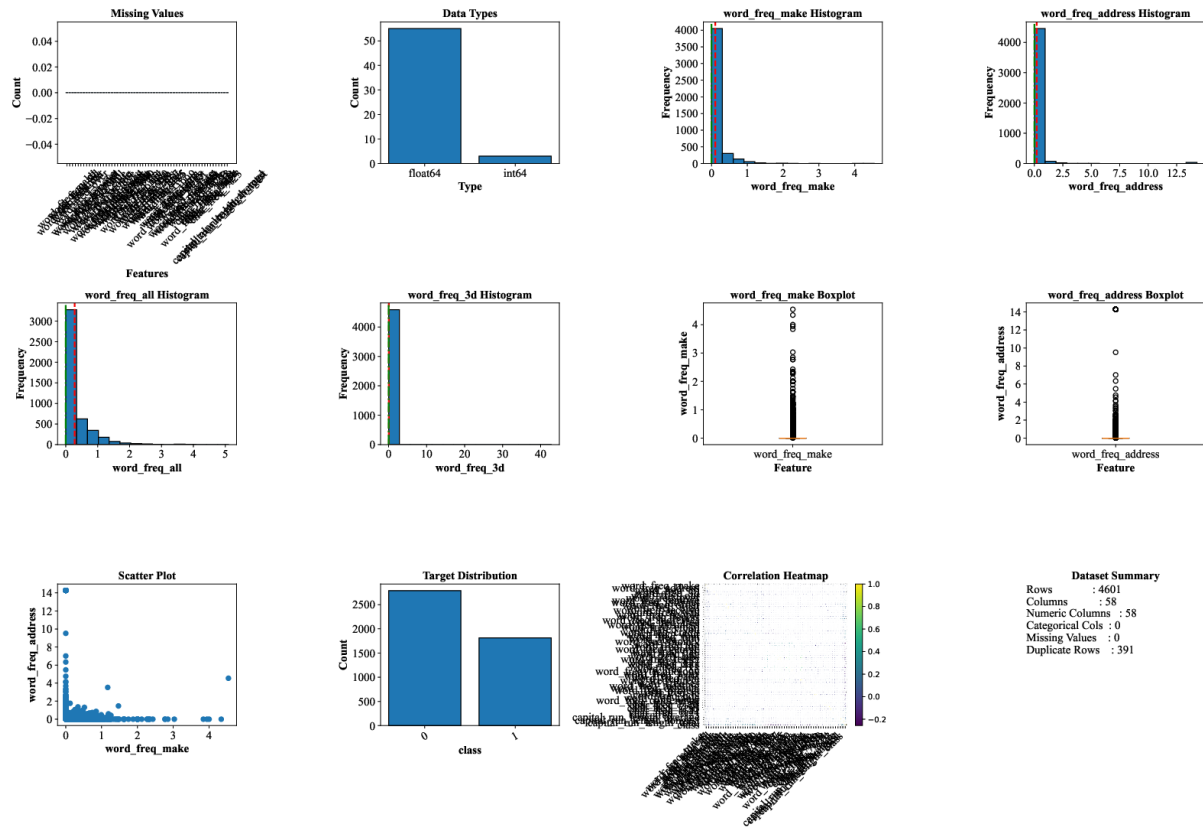


Figure 1: Comprehensive Exploratory Data Analysis (EDA) of the Spambase Dataset

Inference:

The exploratory data analysis shows that the Spambase dataset contains 4,601 email instances described using 57 numerical features representing word frequencies, character frequencies, and capital letter statistics. No missing values are present in the dataset, indicating that no data imputation is required before model training. Although 391 duplicate records exist, the dataset remains sufficiently large for reliable model development after duplicate removal. The class distribution indicates the presence of both spam and non-spam emails, making the dataset suitable for binary classification. The correlation analysis further reveals that several word-frequency features exhibit meaningful relationships with the target variable, suggesting their usefulness for spam detection.

5 Data Preprocessing

The Spambase dataset was prepared for machine learning using a sequence of preprocessing operations implemented through reusable Python functions.

- **Duplicate Removal:**

The dataset was examined for duplicate records. A total of 391 duplicate samples were identified and removed to eliminate redundant observations and improve model generalization.

- **Missing Value Analysis:**

Every feature was inspected for missing values. Since the dataset contains no missing values, no imputation techniques were required.

- **Encoding:**

All predictor variables and the target variable are already represented using numerical values. Consequently, no additional categorical encoding was necessary.

- **Feature Scaling:**

Standardization using the StandardScaler technique was applied before training Gaussian Naïve Bayes and K-Nearest Neighbors. Scaling transforms each feature to have zero mean and unit variance, which improves the performance of distance-based classifiers such as KNN. Multinomial Naïve Bayes and Bernoulli Naïve Bayes were trained using the original non-scaled feature values because these algorithms require non-negative input data.

- **Train-Test Split:**

The processed dataset was divided into training and testing subsets using an 80:20 split. Stratified sampling was employed to preserve the original class distribution in both the training and testing datasets.

- **Feature Selection:**

ANOVA-based SelectKBest feature selection was performed to evaluate feature relevance. Since every feature contributes meaningful information for spam detection, all available features were retained by setting $k = \text{all}$.

6 Machine Learning Algorithms

Four supervised classification algorithms were implemented for email spam detection. Their performance was compared using multiple evaluation metrics, cross-validation, and execution time analysis.

6.1 Gaussian Naive Bayes

Gaussian Naive Bayes assumes that each feature follows a Gaussian (normal) distribution and that all features are conditionally independent given the class label. Although the Spambase dataset contains frequency-based features that are not perfectly Gaussian, this classifier serves as a useful probabilistic baseline.

Mathematical Formulation

For a feature vector $X = (x_1, x_2, \dots, x_n)$, the posterior probability is computed as

$$P(C|X) = \frac{P(C) \prod_{i=1}^n P(x_i|C)}{P(X)}$$

where

$$P(x_i|C) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

The class with the highest posterior probability is selected.

Advantages

- Very fast training and prediction.
- Performs well even with high-dimensional data.
- Simple probabilistic interpretation.

Limitations

- Assumes Gaussian distribution for every feature.
- Independence assumption is often unrealistic.
- Sensitive to feature distributions that deviate significantly from normality.

6.2 Multinomial Naive Bayes

Multinomial Naive Bayes is specifically designed for count or frequency-based data such as text classification. It models the occurrence frequencies of words and is widely used for spam filtering.

Mathematical Formulation

For class C ,

$$P(C|X) \propto P(C) \prod_{i=1}^n P(x_i|C)^{x_i}$$

where

- $P(C)$ is the prior probability of class C .
- $P(x_i|C)$ is the probability of observing feature i in class C .

The classifier predicts the class having the maximum posterior probability.

Advantages

- Well suited for word-frequency based datasets.
- Computationally efficient.
- Performs well for many text classification problems.

Limitations

- Requires non-negative feature values.
- Performance decreases when feature independence assumption is violated.

6.3 Bernoulli Naive Bayes

Bernoulli Naive Bayes models binary-valued features that indicate the presence or absence of attributes. Although the original Spambase features are continuous frequencies, Bernoulli Naive Bayes internally binarizes the values based on a threshold.

Mathematical Formulation

The likelihood for each feature is

$$P(x_i|C) = P_i^{x_i}(1 - P_i)^{1-x_i}$$

where

- $x_i \in \{0, 1\}$.
- P_i represents the probability that feature i is present for class C .

The posterior probability is computed using Bayes' theorem and the class with the highest probability is selected.

Advantages

- Suitable for binary text features.
- Robust for sparse datasets.
- Simple and computationally efficient.

Limitations

- Ignores feature frequency information.
- Less suitable when feature counts contain significant information.

6.4 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric supervised learning algorithm that classifies a new sample according to the majority class among its nearest neighboring training samples. Since KNN relies on distance calculations, feature standardization was performed before training.

Mathematical Formulation

The Euclidean distance between two samples is

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

The predicted class is determined by majority voting among the k nearest neighbors.

Advantages

- Easy to understand and implement.
- No explicit model training is required.
- Effective when similar samples belong to the same class.

Limitations

- Prediction becomes slower for large datasets.
- Sensitive to irrelevant features and feature scaling.
- Choosing an appropriate value of k is important.

7 Experimental Setup

The experiment was implemented in Python using reusable machine learning modules developed during the laboratory sessions. The implementation follows the prescribed Anaconda environment and employs Scikit-Learn for model training, hyperparameter tuning, and evaluation.

Table 2: Experimental Setup

Component	Specification
Python Version	Python 3.12
Scikit-Learn Version	1.9.0
IDE	Jupyter Notebook
Operating System	Ubuntu/Linux (Anaconda Environment)
Processor	Intel(R) Core(TM) i7-10610U CPU @ 1.80 GHz
Memory	16 GB RAM
System Type	64-bit Operating System, x64-based Processor
Dataset	Spambase
Training Samples	3,759
Testing Samples	842

8 Hyperparameter Selection

Different hyperparameters were selected according to the characteristics of each classification algorithm. Grid Search and Randomized Search were additionally performed to identify the optimal KNN configuration.

Table 3: Model Hyperparameters

Parameter	Value
GaussianNB	Default Parameters
MultinomialNB	Default Parameters
BernoulliNB	Default Parameters
Feature Scaling	StandardScaler (GaussianNB and KNN)
Feature Selection	ANOVA (SelectKBest)
Selected Features	All Features ($k = all$)
KNN (k)	5
Distance Metric	Euclidean
GridSearchCV	5-fold Cross Validation
RandomizedSearchCV	10 Iterations

Grid Search evaluated combinations of different values of k , distance metrics, weighting strategies, and search algorithms. The best configuration obtained was:

- Algorithm: Auto
- Distance Metric: Manhattan
- Number of Neighbors: 11
- Weight Function: Distance
- Cross Validation Accuracy: 91.12%

Randomized Search explored a subset of the hyperparameter space and identified the following configuration:

- Algorithm: Ball Tree
- Distance Metric: Manhattan
- Number of Neighbors: 3
- Weight Function: Uniform
- Cross Validation Accuracy: 90.14%

9 Results

The performance of the four classifiers was evaluated using Accuracy, Precision, Recall, F1-score, and ROC–AUC. The results obtained on the testing dataset are summarized in Table 4.

Table 4: Performance Comparison of Classification Models

Algorithm	Accuracy	Precision	Recall	F1-score	ROC–AUC
Gaussian Naive Bayes	0.8290	0.8616	0.8290	0.8306	0.9471
Multinomial Naive Bayes	0.7886	0.7869	0.7886	0.7864	0.8342
Bernoulli Naive Bayes	0.8943	0.8940	0.8943	0.8940	0.9609
K-Nearest Neighbors	0.9109	0.9111	0.9109	0.9104	0.9548

Inference:

Among all the evaluated classifiers, K-Nearest Neighbors achieved the highest classification accuracy of 91.09%, followed closely by Bernoulli Naive Bayes with 89.43%. Multinomial Naive Bayes produced the lowest performance because the Spambase dataset does not completely satisfy its feature assumptions. Gaussian Naive Bayes achieved competitive ROC–AUC despite a comparatively lower classification accuracy. Overall, KNN demonstrated the best balance between precision, recall, and overall classification performance.

9.1 Performance of KNN for Different Values of k

The performance of KNN was analyzed for multiple neighborhood sizes.

Table 5: Performance of KNN for Different Values of k

k	Accuracy	Precision	Recall	F1-score
1	0.9050	0.8832	0.8780	0.8806
3	0.9026	0.8944	0.8571	0.8754
5	0.9109	0.9143	0.8571	0.8848
7	0.9086	0.9164	0.8482	0.8810
9	0.9026	0.9178	0.8304	0.8719
11	0.9014	0.9121	0.8333	0.8709

Inference:

The classifier performed consistently across different neighborhood sizes. The highest testing accuracy was obtained for $k = 5$, which was therefore selected as the default KNN configuration. Increasing the value of k beyond five slightly reduced recall while maintaining similar precision values. This indicates that moderate neighborhood sizes provide a good trade-off between bias and variance.

9.2 GridSearchCV Results

Grid Search was used to evaluate all combinations of selected hyperparameters for KNN.

Table 6: Best Hyperparameters Obtained Using GridSearchCV

Parameter	Best Value
Algorithm	Auto
Distance Metric	Manhattan
Number of Neighbors	11
Weight Function	Distance
Cross Validation Accuracy	0.9112

Inference:

Grid Search evaluated every possible hyperparameter combination within the specified search space. The best configuration employed Manhattan distance with distance-weighted voting and eleven nearest neighbors. This configuration achieved the highest average cross-validation accuracy of 91.12%, indicating good generalization capability.

9.3 RandomizedSearchCV Results

Randomized Search was also performed to identify suitable hyperparameter combinations with lower computational cost.

Table 7: Best Hyperparameters Obtained Using RandomizedSearchCV

Parameter	Best Value
Algorithm	Ball Tree
Distance Metric	Manhattan
Number of Neighbors	3
Weight Function	Uniform
Cross Validation Accuracy	0.9014

Inference:

Randomized Search explored only a subset of all possible combinations. Although it required significantly fewer evaluations than Grid Search, the obtained accuracy was slightly lower. The results demonstrate that Randomized Search provides a good compromise between computational efficiency and predictive performance.

9.4 KD-Tree and Ball-Tree Comparison

The two nearest-neighbor search structures supported by Scikit-Learn were compared.

Table 8: KD-Tree and Ball-Tree Performance Comparison

Algorithm	Accuracy	Training Time (s)	Prediction Time (s)
KD-Tree	0.9109	0.0302	0.2464
Ball-Tree	0.9109	0.0165	0.3901

Inference:

Both search structures achieved identical classification accuracy. Ball-Tree required less training time, whereas KD-Tree provided faster prediction performance on the testing dataset. The results indicate that the choice between the two search structures mainly affects computational efficiency rather than predictive accuracy.

9.5 Cross Validation Results

Five-fold cross validation was performed to evaluate model generalization.

Table 9: Five-Fold Cross Validation Accuracy

Fold	Gaussian NB	Best KNN
1	0.8131	0.8902
2	0.8383	0.8961
3	0.8368	0.9021
4	0.8247	0.9123
5	0.8009	0.9004
Average	0.8227	0.9002

Inference:

KNN consistently outperformed Gaussian Naive Bayes across all five folds. The small variation in fold-wise accuracy indicates stable model performance and good generalization capability. Cross-validation confirms that KNN remains the best-performing classifier for the Spambase dataset.

9.6 Training and Prediction Time Comparison

The computational efficiency of all classifiers was evaluated by measuring training and prediction times.

Table 10: Training and Prediction Time Comparison

Algorithm	Training Time (s)	Prediction Time (s)
Gaussian Naive Bayes	0.0105	0.0018
Multinomial Naive Bayes	0.0080	0.0009
Bernoulli Naive Bayes	0.0164	0.0027
K-Nearest Neighbors	0.0051	0.0491

Inference:

KNN required the least training time because it stores the training instances without constructing an explicit model. However, its prediction time was significantly higher than that of the Naive Bayes classifiers due to the computation of distances to neighboring samples during inference. The Naive Bayes classifiers trained quickly and also produced very fast predictions, making them computationally efficient for real-time applications.

10 Result Visualization

The performance of the implemented classifiers was further analyzed using confusion matrices, ROC curves, and Precision–Recall curves. These visualizations provide deeper insight into the classification capability of each model beyond the numerical performance metrics.

10.1 Exploratory Data Analysis

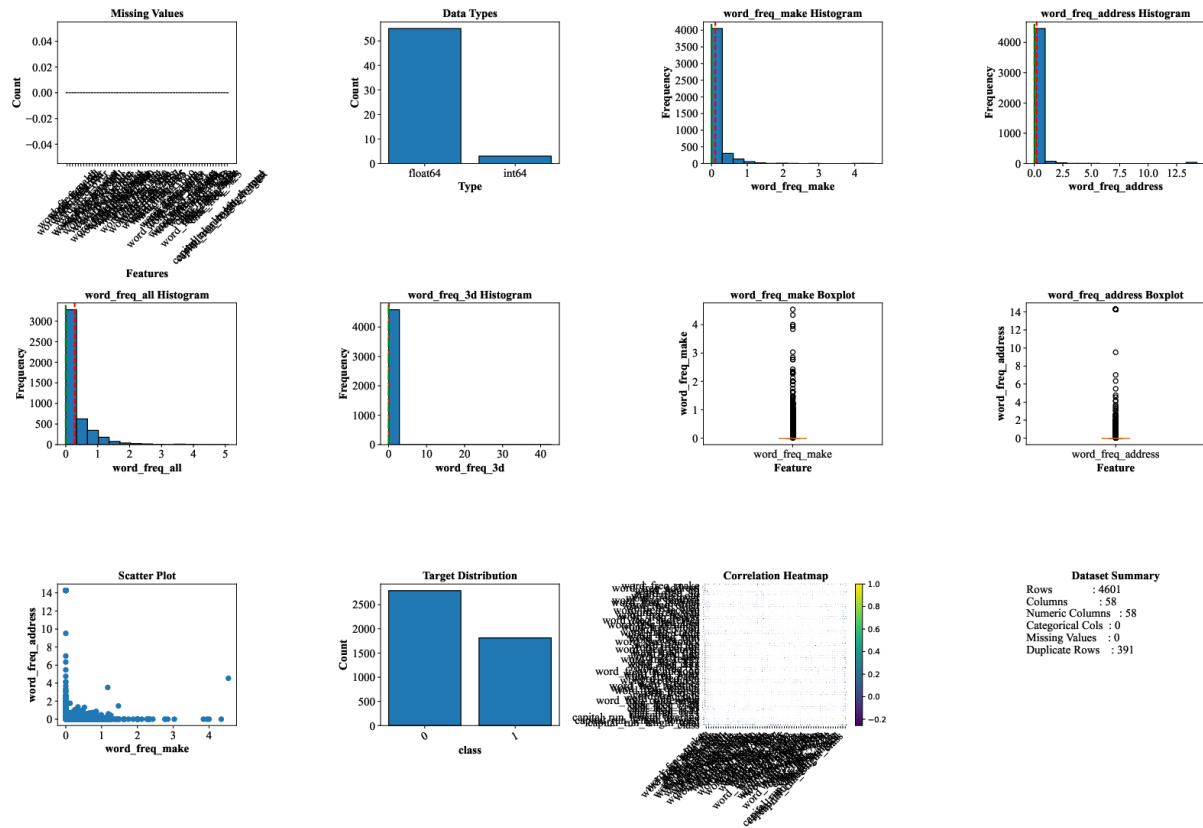


Figure 2: Comprehensive Exploratory Data Analysis of the Spambase Dataset

Inference:

The EDA summary shows that the Spambase dataset contains no missing values and consists entirely of numerical attributes. The class distribution indicates a moderate imbalance between spam and non-spam emails. Several word-frequency and character-frequency features exhibit skewed distributions with many zero values. The correlation analysis suggests that only a subset of the features are strongly correlated, making the dataset suitable for machine learning classification.

10.2 Gaussian Naive Bayes

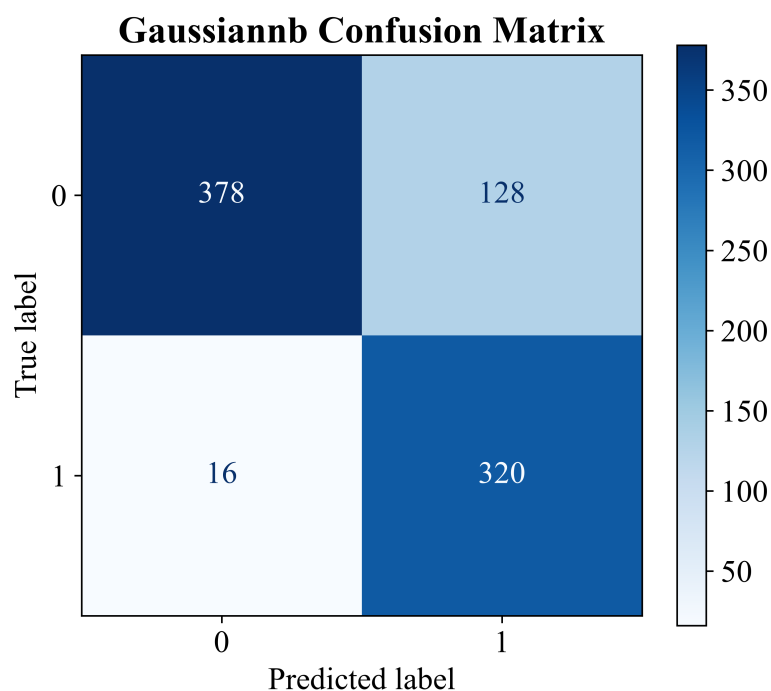


Figure 3: Confusion Matrix of Gaussian Naive Bayes

Inference:

Gaussian Naive Bayes correctly classified the majority of both spam and non-spam emails. However, several legitimate emails were incorrectly classified as spam, increasing the number of false positives. Despite these errors, the classifier maintained good overall performance. The confusion matrix demonstrates that Gaussian Naive Bayes provides a reasonable baseline classifier.

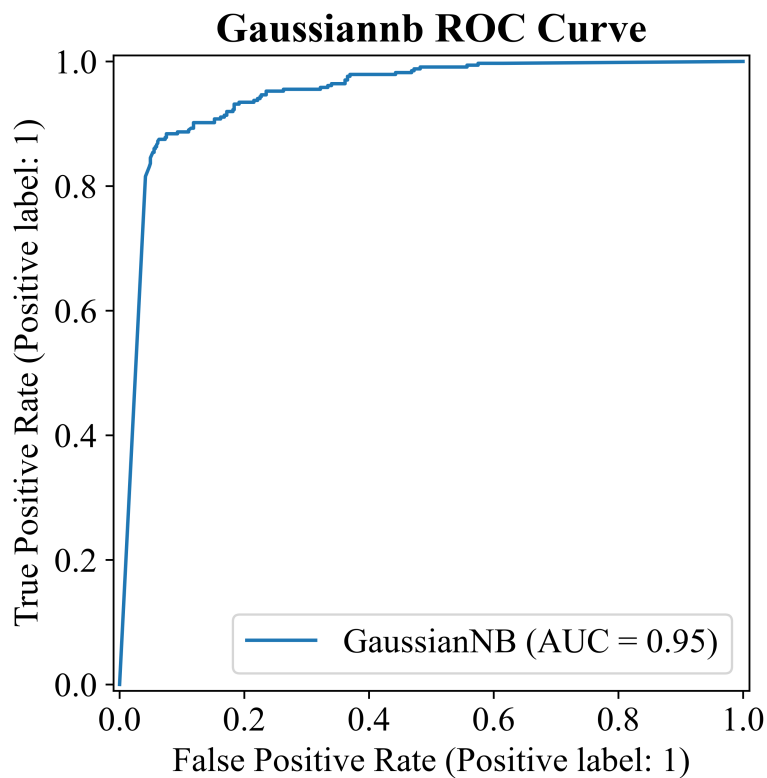


Figure 4: ROC Curve of Gaussian Naive Bayes

Inference:

The ROC curve lies close to the upper-left corner, producing an ROC-AUC value of approximately 0.95. This indicates excellent discrimination between spam and non-spam emails. The classifier achieves high true-positive rates while maintaining relatively low false-positive rates.

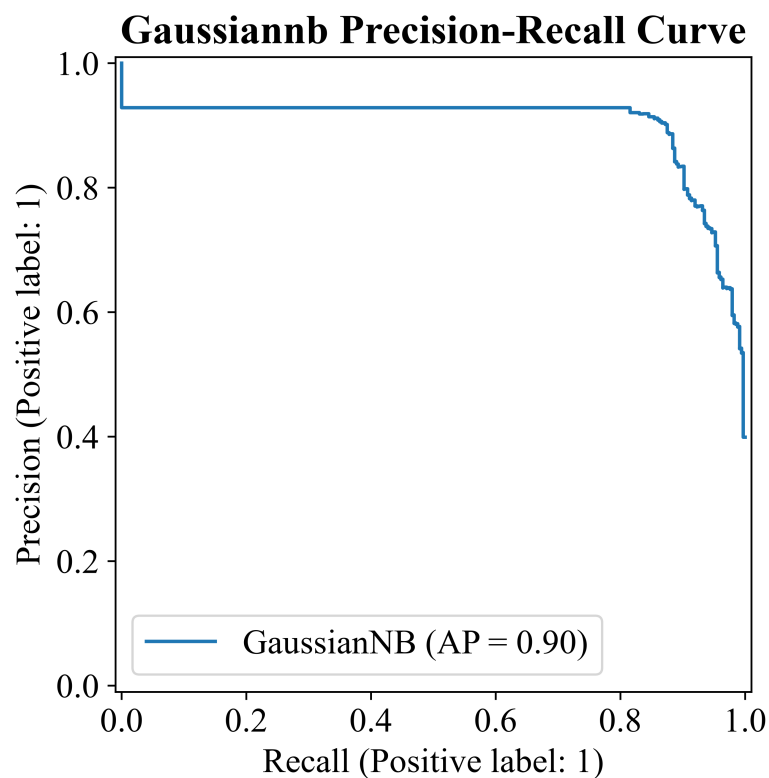


Figure 5: Precision–Recall Curve of Gaussian Naive Bayes

Inference:

The Precision–Recall curve demonstrates that Gaussian Naive Bayes maintains good precision across a wide range of recall values. The gradual decline in precision at higher recall indicates the expected trade-off between identifying more spam emails and increasing false positives.

10.3 Multinomial Naive Bayes

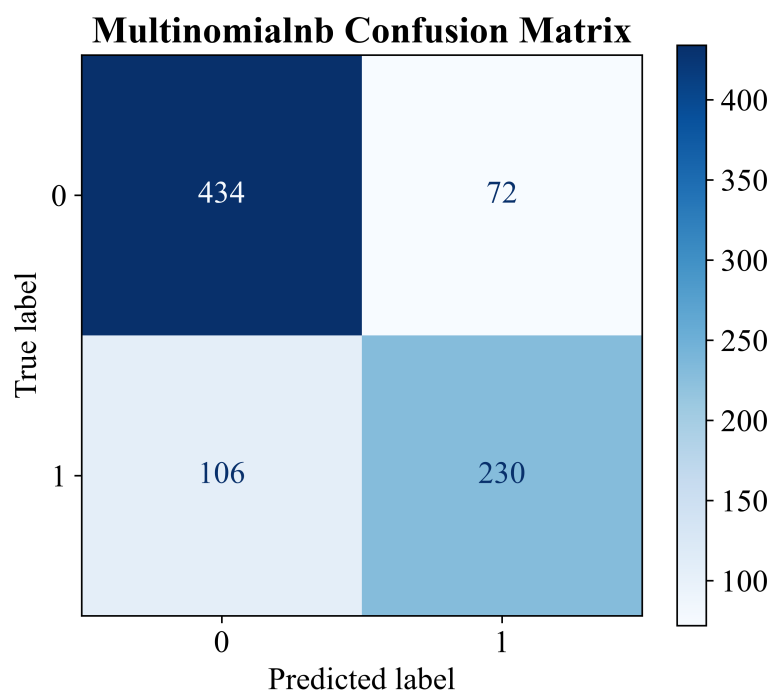


Figure 6: Confusion Matrix of Multinomial Naive Bayes

Inference:

Multinomial Naive Bayes produced a larger number of false negatives compared with the other classifiers. Consequently, several spam emails were classified as legitimate emails. This reduced the recall and overall classification accuracy, indicating that the model assumptions do not perfectly match the feature distribution of the Spambase dataset.

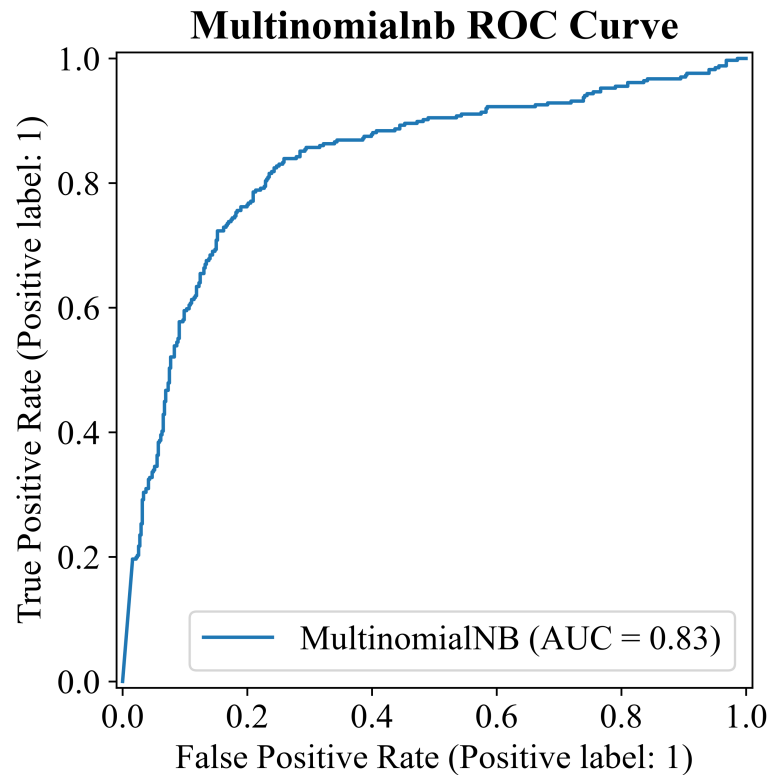


Figure 7: ROC Curve of Multinomial Naive Bayes

Inference:

The ROC curve remains above the random classifier line but is farther from the ideal upper-left corner compared to the other models. The ROC-AUC value of approximately 0.83 reflects comparatively weaker discrimination capability between the two classes.

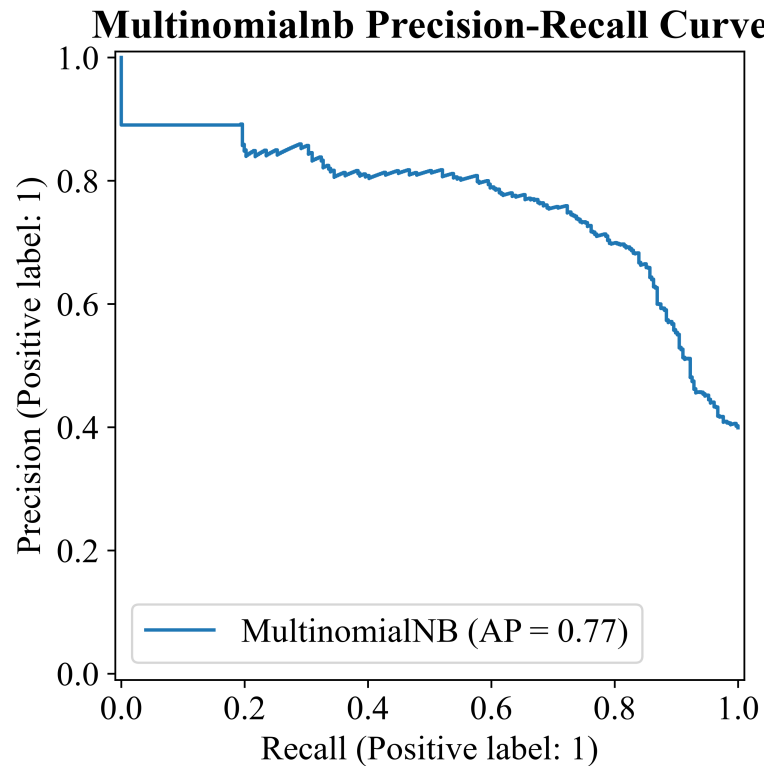


Figure 8: Precision–Recall Curve of Multinomial Naive Bayes

Inference:

The Precision–Recall curve shows a noticeable reduction in precision as recall increases. This behavior indicates that the classifier struggles to identify all spam emails while maintaining a low false-positive rate. Overall, it performs less effectively than the remaining classifiers.

10.4 Bernoulli Naive Bayes

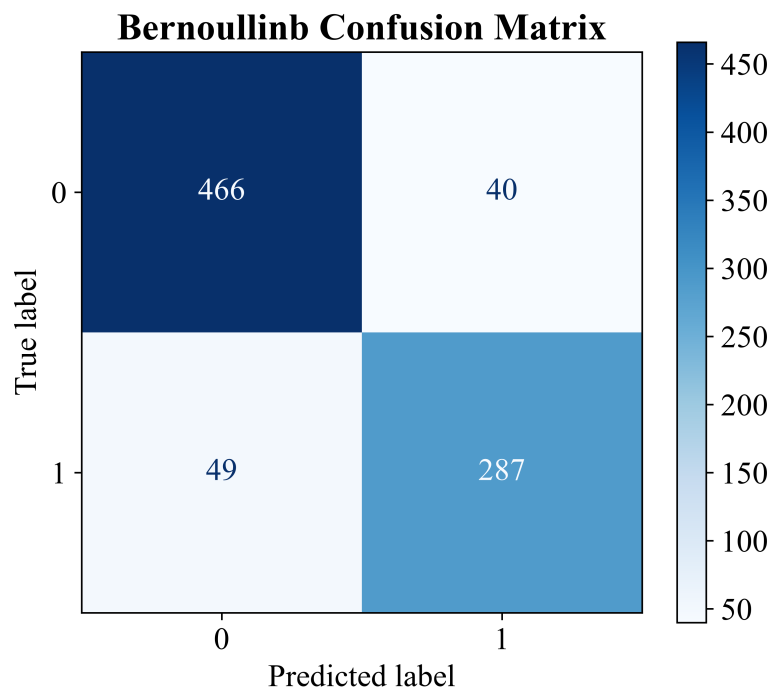


Figure 9: Confusion Matrix of Bernoulli Naive Bayes

Inference:

Bernoulli Naive Bayes correctly classified most spam and non-spam emails with relatively few classification errors. The confusion matrix indicates balanced performance across both classes. The classifier significantly outperformed Multinomial Naive Bayes on this dataset.

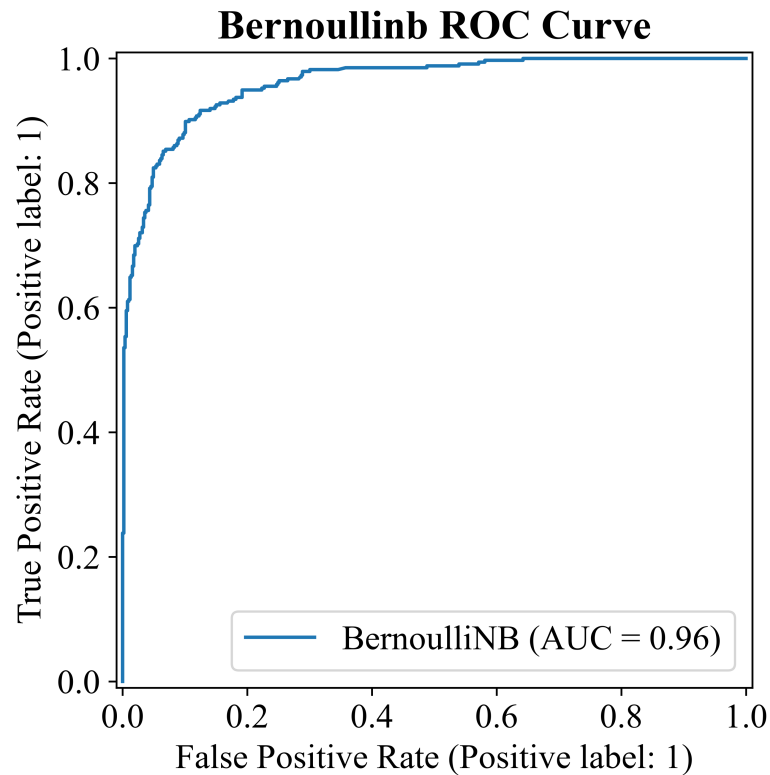


Figure 10: ROC Curve of Bernoulli Naive Bayes

Inference:

The ROC curve is positioned very close to the upper-left corner and achieves an ROC-AUC value exceeding 0.96. This demonstrates excellent discrimination capability and confirms that Bernoulli Naive Bayes is highly effective for this binary classification task.

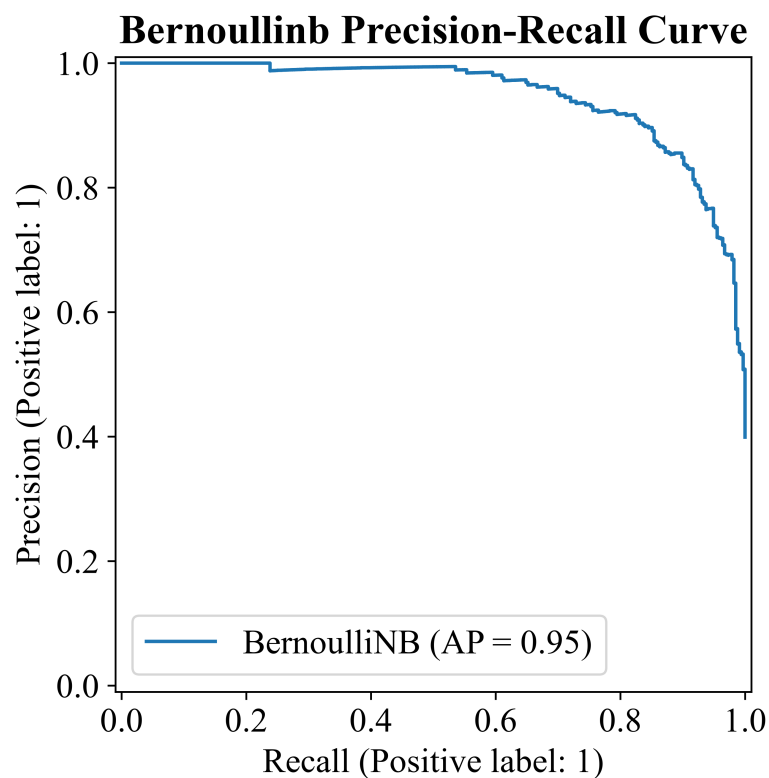


Figure 11: Precision–Recall Curve of Bernoulli Naive Bayes

Inference:

The Precision–Recall curve remains consistently high throughout most threshold values. High precision and high recall indicate that the classifier successfully identifies spam emails while minimizing false alarms.

10.5 K-Nearest Neighbors

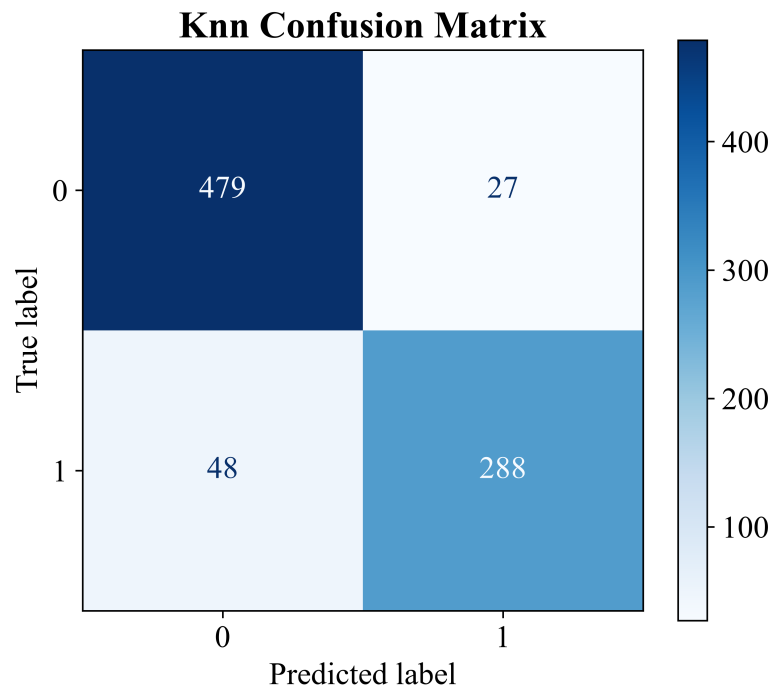


Figure 12: Confusion Matrix of K-Nearest Neighbors

Inference:

The KNN classifier correctly classified the majority of both spam and non-spam emails, resulting in the highest overall accuracy among all evaluated models. The confusion matrix contains relatively few false positives and false negatives, demonstrating balanced classification performance.

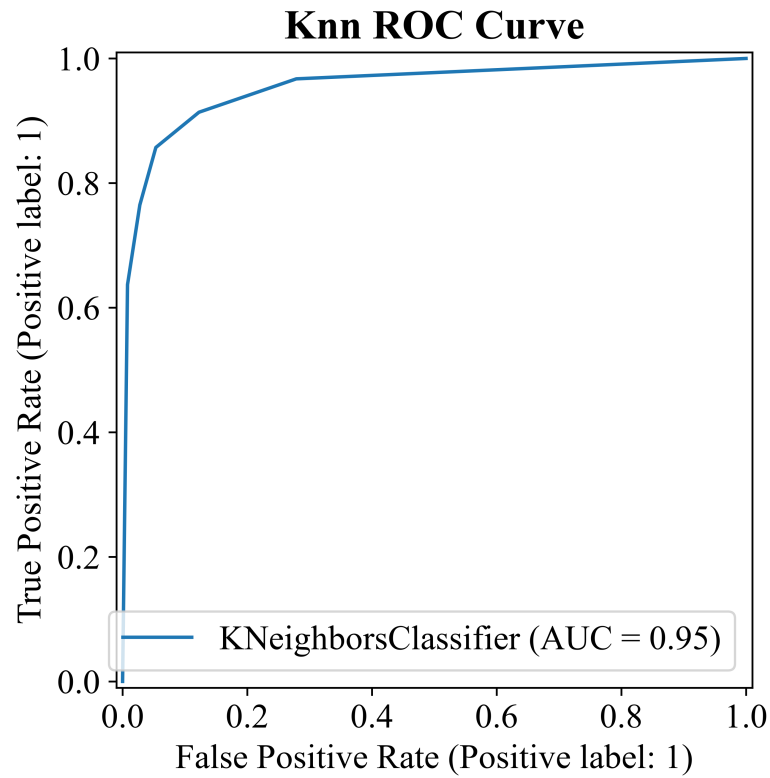


Figure 13: ROC Curve of K-Nearest Neighbors

Inference:

The ROC curve illustrates excellent separation between the two classes, with an ROC-AUC value of approximately 0.95. The classifier maintains a high true-positive rate while keeping false positives comparatively low across different classification thresholds.

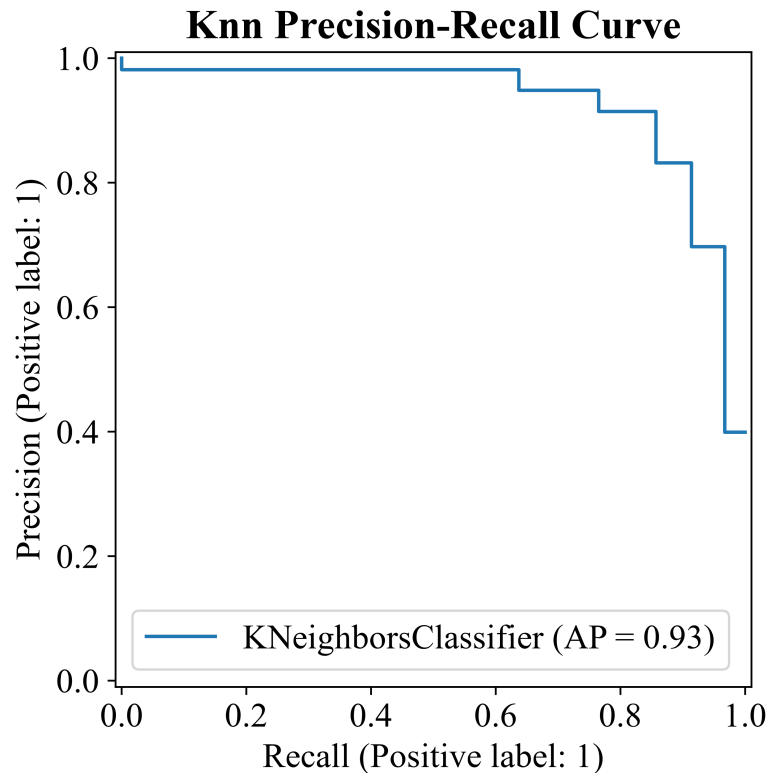


Figure 14: Precision–Recall Curve of K-Nearest Neighbors

Inference:

The Precision–Recall curve remains consistently high, indicating strong predictive performance over a wide range of thresholds. The classifier achieves an effective balance between precision and recall, making it highly suitable for spam detection where both false positives and false negatives are important.

11 Discussion

This experiment demonstrated the application of probabilistic and instance-based machine learning algorithms for email spam classification using the Spambase dataset. Exploratory Data Analysis confirmed that the dataset is clean, contains no missing values, and consists of numerical word-frequency and character-frequency attributes suitable for supervised learning. Since the dataset contains continuous-valued features with varying scales, feature standardization was applied before training Gaussian Naive Bayes and K-Nearest Neighbors, while the original non-negative feature values were retained for Multinomial and Bernoulli Naive Bayes.

Among the evaluated classifiers, K-Nearest Neighbors achieved the highest testing accuracy of 91.09%, demonstrating superior classification capability on the Spambase dataset. Bernoulli Naive Bayes also performed well with an accuracy of 89.43%, indicating that binary feature modeling is effective for spam detection. Gaussian Naive Bayes achieved satisfactory performance despite its assumption that features follow Gaussian distributions. Multinomial Naive Bayes produced the lowest classification accuracy because the feature distribution in the Spambase dataset does not completely satisfy its underlying assumptions.

Hyperparameter tuning using GridSearchCV and RandomizedSearchCV further improved the understanding of KNN behavior. Grid Search identified Manhattan distance with distance-weighted voting and eleven neighbors as the best configuration during cross-validation, whereas Randomized Search obtained a competitive solution with fewer parameter evaluations. Five-fold cross validation demonstrated that KNN consistently outperformed Gaussian Naive Bayes across all folds, confirming its strong generalization ability.

The execution time analysis highlighted the computational trade-offs among the classifiers. KNN required minimal training time because it stores the training instances directly without constructing an explicit model. However, it incurred the highest prediction time since distance calculations must be performed for every testing sample. In contrast, the Naive Bayes classifiers trained rapidly and produced extremely fast predictions, making them attractive for real-time email filtering applications.

Overall, the experiment illustrates the importance of selecting an appropriate classifier based on both predictive performance and computational efficiency. While KNN achieved the best classification accuracy, the Naive Bayes classifiers provide faster inference and lower computational overhead. The reusable implementation developed during this experiment can be extended to larger email datasets and other binary text classification problems.

12 Conclusion

This experiment successfully implemented and compared four supervised classification algorithms for email spam detection using the Spambase dataset. The complete machine learning workflow included dataset exploration, preprocessing, feature selection, model training, hyperparameter tuning, cross-validation, performance evaluation, and visualization.

The comparative study showed that K-Nearest Neighbors achieved the highest classification accuracy of 91.09%, followed by Bernoulli Naive Bayes with an accuracy of 89.43%. Gaussian Naive Bayes provided competitive ROC-AUC performance, whereas Multinomial Naive Bayes achieved comparatively lower accuracy because of the characteristics of the dataset. Hyperparameter tuning using GridSearchCV and RandomizedSearchCV demonstrated the importance of selecting suitable neighborhood size, distance metric, and weighting strategy for KNN.

Cross-validation confirmed that KNN generalized consistently across different training folds. The execution time analysis also revealed that although KNN requires very little training time, its prediction time is higher than that of the Naive Bayes classifiers. Consequently, the choice of classifier depends on the balance between predictive accuracy and computational efficiency required by the application.

Overall, the experiment provided practical experience in implementing reusable machine learning modules for classification tasks and reinforced the significance of preprocessing, model selection, hyperparameter optimization, and comprehensive performance evaluation in developing reliable spam detection systems.

13 References

1. Scikit-learn Developers, *Scikit-learn Documentation*. Available: <https://scikit-learn.org/stable/>
2. Dua, D., and Graff, C., *UCI Machine Learning Repository: Spambase Data Set*. Available: <https://archive.ics.uci.edu/ml/datasets/spambase>
3. Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3rd ed., O'Reilly Media, 2023.
4. Mitchell, T. M., *Machine Learning*. McGraw-Hill Education, 1997.
5. Bishop, C. M., *Pattern Recognition and Machine Learning*. Springer, 2006.